

Architecting the Post-GPU Era: Translating AI Kernels to Spatial Dataflow for FPGA and ASIC Deployment

The rapid proliferation of artificial intelligence has historically been inextricably linked to the architectural evolution of Graphics Processing Units (GPUs). Designed for massive thread-level parallelism, GPUs excel at dense matrix multiplications when fed with large batch sizes, effectively hiding memory latency behind thousands of concurrent execution threads¹. However, as the industry pivots toward real-time, ultra-low-latency inference—particularly for autoregressive Large Language Models (LLMs) and subquadratic sequence models operating at a batch size of one—the GPU's Single Instruction, Multiple Thread (SIMT) architecture encounters severe physical and architectural bottlenecks².

To overcome these limitations, hardware architects are increasingly deploying custom spatial dataflow architectures on Field-Programmable Gate Arrays (FPGAs) and transitioning these proven designs to Application-Specific Integrated Circuits (ASICs)³. This report provides an exhaustive methodology for translating high-level GPU kernels (authored in domain-specific languages like Triton) into optimized Register Transfer Level (RTL) implementations. It examines the obsolescence of legacy "soft CPU" paradigms, defines the theoretical limits of modern spatial architectures, proposes a rigorous pre-silicon verification methodology, and outlines the physical design safeguards required before committing an FPGA prototype to a multi-million-dollar ASIC tapeout.

1. The Obsolescence of Soft CPUs in Modern AI Workloads

Historically, migrating complex logic to an FPGA required wrapping algorithmic kernels around instantiated "soft CPUs," such as Xilinx's MicroBlaze or Intel's Nios. These soft processors were necessary to handle the control flow, orchestrate memory transfers, and manage state machines, effectively acting as an embedded microcontroller overseeing hardware accelerators³. The developer would write C code for the soft CPU, which would then issue memory-mapped commands to discrete hardware blocks.

In the modern AI paradigm, this soft-CPU overhead is largely obsolete. The control flow is instead embedded directly into distributed, highly pipelined dataflow state machines that orchestrate streams of tensors through custom processing elements⁵. Unlike von Neumann architectures that rely on program counters, instruction fetches, and centralized register files, spatial dataflow architectures are inherently decentralized. The execution graph itself becomes

the hardware⁷.

Data moves continuously from memory interfaces into the computational array via AXI4-Stream interfaces, relying entirely on asynchronous handshakes (valid/ready signals) to propagate execution⁷. The elimination of instruction decoding and central control logic liberates vast amounts of silicon area and power, allowing the FPGA or ASIC to dedicate a significantly higher proportion of its transistor budget directly to multiply-accumulate (MAC) operations and local Static Random-Access Memory (SRAM)⁵.

2. Defining the Architectural Limits: GPUs vs. Spatial Dataflow

Before proposing a specific design methodology, it is critical to identify the fundamental limits of the competing architectures. The primary constraint in modern AI inference—particularly for LLMs—is not arithmetic throughput (FLOPs), but rather memory bandwidth and latency².

2.1 The Memory Wall at Batch-1 Inference

When a GPU performs autoregressive decoding (generating text one token at a time), it operates at a batch size of one. Architectures like Gated DeltaNet (GDN) or standard Multi-Head Self-Attention (MHSA) exhibit arithmetic intensities well below 1 FLOP/Byte during this decode phase². To sustain peak performance, an architecture requires high operational intensity, meaning a large amount of computation must be performed for every byte of data fetched from memory.

On an NVIDIA H100 GPU, the 50 MB L2 cache is a hardware-managed caching hierarchy that cannot guarantee the persistence of a model's key-value (KV) cache or recurrent state matrices across successive kernel invocations². Consequently, the GPU must round-trip the entire state through its external High Bandwidth Memory (HBM) for every single token generated. This mandatory off-chip data movement renders the GPU profoundly memory-bound². Despite possessing a peak theoretical compute limit of 51.2 TFLOP/s, the H100's HBM bandwidth of 2.0 TB/s chokes the Tensor Cores, leaving them severely underutilized².

FPGAs and ASICs subvert this limit through explicit, software-defined spatial dataflow. Because an FPGA's internal memory (Block RAM and UltraRAM) can be statically allocated by the designer, the entire recurrent state of a model (e.g., the 2 MB state of a GDN layer) can be held persistently on-chip². By eliminating the off-chip HBM round-trip, the spatial architecture converts a memory-bound workload into a compute-bound workload. Evaluations demonstrate that this persistent-state approach on an FPGA achieves per-token latencies of 63 microseconds—outpacing flagship GPUs by factors of 4.5x while consuming roughly 10 Watts on-chip, yielding up to a 60x improvement in energy efficiency².

2.2 Reconfigurable Hardware Substrates: The DSP58 Evolution

When targeting modern architectures, such as the AMD Versal Adaptive Compute Acceleration Platform (ACAP), the compiler must map computations to advanced silicon structures. The

Versal architecture supersedes legacy DSP48 slices by integrating DSP58 blocks, which natively support INT8 vector modes alongside floating-point operations¹⁴. In highly optimized spatial designs, a single DSP58 can execute multiple MAC operations per cycle using SIMD (Single Instruction, Multiple Data) partitioning. By configuring these DSP slices in a systolic chain, engineers can form robust multiply-accumulate engines without exhausting the general-purpose LUT routing fabric¹⁵. At peak utilization, this enables up to 185 INT8 dense TOPS on high-end Versal AI Edge devices¹⁶. The synthesis tools must be rigorously guided to infer these specific primitives rather than synthesizing expensive MAC units out of general-purpose logic.

Architectural Feature	GPU (e.g., NVIDIA H100)	Spatial Dataflow (e.g., Versal FPGA/ASIC)
Execution Paradigm	SIMT (Thread-level parallelism)	Pipeline/Spatial Dataflow (Hardware streams)
Memory Hierarchy	Hardware-managed caches (L1/L2)	Software-defined distributed SRAM (BRAM/URAM)
Optimal Workload	Large-batch dense matrix multiplication	Batch-1 latency, sparse graphs, custom bit-widths
Control Overhead	High (Warp schedulers, instruction fetch)	Near Zero (Valid/Ready AXI handshakes)
On-Chip Persistence	Volatile across kernel invocations	Persistent (Eliminates off-chip memory walls)

3. The Methodology: Translating GPU Kernels to Hardware

Modern machine learning compilers, such as TorchInductor, rely heavily on Triton to lower PyTorch models into optimized GPU kernels¹⁸. Triton provides a Python-based DSL (Domain Specific Language) that abstracts memory coalescing, shared memory management, and thread-block scheduling, hiding the underlying hardware complexity from the developer²⁰. While highly effective for NVIDIA and AMD GPUs, Triton's execution model intrinsically assumes a specific hardware topology: a central global memory, a shared L2 cache, and streaming multiprocessors (SMs)¹⁰. FPGAs and ASICs lack this topology. Attempting to map a Triton kernel directly to an FPGA using traditional High-Level Synthesis (HLS) tools (e.g., Vitis HLS) often results in catastrophic

performance degradation. Standard HLS tools attempt to emulate GPU thread-block scheduling using sequential C++ semantics, failing to exploit the true spatial potential of the logic fabric²².

3.1 Overcoming HLS Limitations via MLIR

To bridge this semantic chasm, the compilation stack must rely on Multi-Level Intermediate Representation (MLIR), a modular compiler infrastructure that facilitates the progressive lowering of abstractions²⁴. Instead of compiling Triton directly to machine code, frameworks such as ScaleHLS and StreamTensor intercept the compilation at the intermediate representation level⁶.

The translation pipeline enforces a strict hierarchical lowering process to systematically strip away GPU-centric assumptions and synthesize cycle-accurate hardware streams:

1. **Tensor-Level Abstraction:** The PyTorch computational graph is imported via Torch-MLIR into the *tosa* (Tensor Operator Set Architecture) or *linalg* (Linear Algebra) dialects⁶. At this stage, operations are modeled as pure mathematical functions on multi-dimensional arrays, completely devoid of hardware constraints or memory layouts.
2. **Loop and Affine Transformations:** The *linalg* operations are subsequently lowered into the *affine* and *scf* (Structured Control Flow) dialects. Here, high-level matrix multiplications are decomposed into explicit nested loops. Tools like ScaleHLS apply polyhedral loop optimization, loop tiling, and loop unrolling to expose fine-grained parallelism that aligns with the target FPGA's DSP capacity²⁶.
3. **Dataflow and Stream Materialization:** Unlike GPUs, which rely on memory-mapped tensor types accessible via global pointers, FPGAs demand streaming access patterns. The compiler transforms memory-mapped accesses into strictly ordered First-In-First-Out (FIFO) channels. Frameworks introduce specialized dialects (e.g., custom stream types or *hlsccp*) to explicitly encode stream layouts, ensuring that producer and consumer kernels are safely connected via non-blocking AXI4-Stream interfaces⁶.
4. **Hardware Generation via CIRCT:** The final MLIR representation is lowered either to optimized C++ for consumption by vendor HLS tools or, increasingly, directly into FIRRTL (Flexible Intermediate Representation for RTL) using the CIRCT (Circuit IR Compilers and Tools) project²⁸. CIRCT is revolutionizing the methodology because it bypasses the inherent limitations of C++ HLS by directly synthesizing structural RTL. This provides absolute control over clock-cycle-level scheduling, preventing the HLS compiler from inserting pessimistic stalls due to perceived memory dependencies³¹.

3.2 Initiation Intervals and Ping-Pong Buffering

A proposed design must maximize the throughput of the underlying DSP slices. To sustain high utilization, spatial architectures rely heavily on "ping-pong" (double) buffering⁸. In a typical deep learning layer, data loading and computation represent two distinct phases. If a single memory bank is utilized, the processing elements (PEs) must stall while the Direct Memory Access (DMA) engine fetches the next block of data from external memory, degrading performance. Ping-pong buffering partitions the local SRAM into two isolated halves. While the AXI4-Stream interface fills Buffer A with the next tile of activations, the systolic array actively computes using

the data already resident in Buffer B³³. Once the computation finishes, the roles swap seamlessly, completely hiding the memory latency behind the arithmetic operations³⁴. The efficiency of this pipeline is governed by a metric known as the Initiation Interval (II)—the number of clock cycles between the start of successive loop iterations³⁵. In a perfectly pipelined HLS design, $II = 1$, meaning a new computation begins every single clock cycle, maximizing hardware throughput. However, memory dependencies, particularly Write-After-Read (WAR) hazards, often force the HLS compiler to conservatively increase the II²³. Automated frameworks address this by using iterative integer-linear-programming (ILP) to calculate optimal tile sizes, manage FIFO depths, and structurally guarantee $II = 1$ while avoiding dataflow deadlocks across the entire system⁵.

4. High-Level Synthesis vs. Handwritten RTL

A persistent debate in ASIC and FPGA design is the efficacy of HLS versus handwritten RTL (SystemVerilog/VHDL). HLS allows developers to iterate rapidly on algorithm logic, bit-widths, and quantization profiles, synthesizing C++ into RTL in a matter of minutes³⁷. However, delegating microarchitectural decisions to a compiler exacts a penalty. On average, HLS-generated designs consume approximately 20% more logic resources (LUTs and registers) and achieve maximum clock frequencies (F_{max}) that are 10% to 50% lower than expertly hand-coded RTL³⁸. Furthermore, HLS tools are notoriously conservative regarding memory hazards; if the compiler cannot statically prove that two memory accesses will not collide, it inserts pessimistic scheduling logic that degrades the Initiation Interval³⁹. For a methodology targeting eventual ASIC tapeout, a hybrid approach is strongly recommended:

Metric	High-Level Synthesis (C++/MLIR)	Handwritten RTL (SystemVerilog)
Primary Domain	Algorithmic Datapaths, Math Kernels, ML Ops	Control Logic, Interconnects, Arbiters, Custom NoC
Development Speed	Weeks (Rapid iterative exploration)	Months (Slow, meticulous coding)
F_{max} (Clock Speed)	Moderate (Subject to compiler scheduling)	High (Optimized critical paths)
Resource Efficiency	Sub-optimal (~20%	Highly optimized (Minimal

	overhead)	gate waste)
Verification Strategy	C/RTL Co-simulation	Cycle-accurate UVM testbenches

In this hybrid methodology, dense compute kernels (e.g., matrix multiplications, convolutions, activation functions) are generated using HLS or MLIR frameworks like ScaleHLS to rapidly explore the design space for tile sizes and unroll factors²⁵. Conversely, the top-level orchestration, AXI-Stream arbiters, custom network-on-chip (NoC) routers, and highly complex finite state machines (FSMs) are meticulously hand-written in SystemVerilog. This bifurcated approach ensures tight timing closure and absolute control over hardware interfaces while preserving algorithmic agility³⁷.

5. Exhaustive Pre-Silicon Verification: De-risking the Tapeout

FPGAs serve as excellent deployment vehicles for low-to-medium volume applications, but at high volumes (e.g., >100,000 units), the unit cost of a high-end FPGA (\$100 to \$2,000+) becomes economically prohibitive within a bill-of-materials⁴². To achieve economies of scale and maximum power efficiency, the design must be migrated to a custom ASIC.

An ASIC tapeout at advanced nodes (e.g., 5nm or 7nm) carries a Non-Recurring Engineering (NRE) cost ranging from \$5 million to over \$30 million for the mask sets⁴². A single functional bug discovered post-fabrication (necessitating a "silicon respin") can cost millions of dollars and delay product launches by six to nine months. Therefore, testing functionality on an FPGA is insufficient; the RTL must undergo exhaustive, multi-layered pre-silicon verification.

5.1 Formal Verification and SystemVerilog Assertions (SVA)

Traditional simulation only tests the input vectors explicitly provided by the verification engineer; it cannot definitively prove the absence of bugs in obscure corner cases. Formal verification addresses this by mathematically proving that specific design properties hold true under *all* possible legal input combinations⁴⁴.

In custom AI accelerators utilizing spatial dataflow, a primary risk is protocol deadlock within the asynchronous AXI4-Stream handshakes connecting the kernels⁷. If a downstream consumer stops asserting ready while an upstream producer continues asserting valid, the intervening ping-pong buffers will fill, and the entire pipeline will irreversibly stall.

Verification engineers utilize formal tools like Cadence JasperGold or Synopsys VC Formal to perform bounded model checking on the design⁴⁶. The expected behavior is codified using SystemVerilog Assertions (SVA)⁴⁵. To mathematically prove that an arbiter guarantees mutual exclusivity and prevents starvation, an engineer might assert:

- **Mutual Exclusivity:** assert property (@(posedge clk) !(master0_grant &&

```
master1_grant));
```

```
[cite: 48]
```

- **Starvation Freedom:** `assert property (@(posedge clk) (master0_request && !master0_grant) | => ##[1:MAX_CYCLES] master0_grant);`

```
[cite: 48]
```

If a property can be violated under any conceivable state, the formal solver returns a counter-example trace—a waveform showing the exact cycle-by-cycle sequence of events that triggers the failure⁴⁶. Emerging workflows even utilize large language models (LLMs) to automatically generate comprehensive libraries of SVA properties from architectural specifications (e.g., HierSVA), streamlining the formal signoff process⁴⁵.

5.2 Universal Verification Methodology (UVM) and DPI-C Integration

For dynamic functional verification, the industry standard is the Universal Verification Methodology (UVM). UVM provides a robust, class-based, object-oriented framework in SystemVerilog for generating constrained-random stimuli, driving them into the Design Under Test (DUT), and monitoring the outputs via scoreboards⁴⁹.

When migrating AI GPU kernels to ASICs, the original PyTorch model serves as the ultimate golden reference. Using the Direct Programming Interface (DPI-C), verification engineers embed C/C++ or Python wrappers directly into the SystemVerilog testbench⁴⁹. This architecture allows the UVM sequence generator to pass randomly generated input tensors simultaneously to the RTL simulator and the live PyTorch model. The outputs of the RTL are then rigorously checked against the PyTorch reference down to exact quantized integer values, ensuring perfect mathematical equivalence before tapeout⁵⁰.

5.3 Hardware Emulation: Scaling to System-Level Workloads

Software simulators (like VCS or Xcelium) execute RTL sequentially on host CPUs. For a complex AI accelerator comprising millions of gates, simulation speeds drop to the Hertz (Hz) range, making it practically impossible to run full software stacks or extensive ML inference workloads (such as generating hundreds of tokens in an LLM)⁵¹.

To boot firmware and validate true system-level integration, the RTL is mapped to hardware emulators such as Cadence Palladium or Synopsys ZeBu⁵². Unlike FPGAs, which require lengthy logic synthesis and place-and-route (P&R) cycles, emulators utilize massively parallel arrays of custom processors to execute the RTL directly⁵². Palladium achieves operating speeds of 1 to 1.5 MHz—roughly 1000x faster than software simulation—while maintaining full cycle-accuracy and providing complete waveform visibility into every net in the hierarchy⁵². This enables the execution of deep, realistic workloads to uncover deep-state bugs that would never manifest in truncated simulation runs⁵⁴.

6. The FPGA-to-ASIC Migration Protocol: Physical Design Safeguards

Once the RTL is functionally frozen and formally verified, the physical design (backend) phase commences. Migrating from an FPGA to an ASIC is not merely a matter of transferring RTL to a foundry; FPGAs conceal immense physical complexity that must be manually engineered by the ASIC design team.

6.1 Power Distribution Network (PDN) and IR Drop Analysis

FPGAs are manufactured with over-provisioned, hardened power grids designed to handle generic, worst-case scenarios. In a custom ASIC, the physical design team must build the Power Distribution Network (PDN) entirely from scratch, carefully sizing the metal layers and vias to balance routing congestion against electrical resistance⁵⁵.

As transistor densities scale down to 5nm or 7nm, operating voltages drop (often below 0.7V), and local current densities skyrocket, particularly within dense MAC arrays of an AI

accelerator⁵⁵. As current (I) flows through the resistive (R) power straps, it induces a voltage drop known as *IR Drop*⁵⁵. If the local supply voltage sags below a critical threshold during a burst of AI computation (dynamic IR drop), transistor switching slows down. This induces setup-time violations, data corruption, or catastrophic functional failure, particularly at high temperatures⁵⁵. Tools such as Ansys RedHawk or Cadence Voltus are employed to simulate transient current demands and identify voltage hotspots on the floorplan, allowing engineers to reinforce the PDN before tapeout⁵⁵.

6.2 Design for Testability (DFT) and Scan Insertion

FPGAs are tested for manufacturing defects by the vendor before sale. For a custom ASIC, the design team bears the responsibility of proving that the fabricated silicon matches the RTL and contains no physical defects (e.g., shorts or opens caused by microscopic dust in the lithography process)⁵⁸.

This is achieved through Design for Testability (DFT) methodologies. The most prevalent technique is scan insertion, where standard flip-flops are replaced with scan flip-flops that can be daisy-chained into long shift registers⁵⁹. During post-fabrication testing, Automatic Test Pattern Generation (ATPG) vectors are shifted through these chains to exercise combinatorial logic paths. However, DFT insertion inherently introduces multiplexers into the data path, slightly increasing the die area and degrading timing margins⁶¹. Achieving timing closure after scan insertion is widely considered one of the most challenging phases of the ASIC backend flow⁶³.

6.3 Tapeout, Metal ECOs, and Spare Gates

The final signoff process—involving rigorous checks for static timing, power, design rules (DRC), and layout versus schematic (LVS)—culminates in the "tapeout," where the design database (GDSII) is transmitted to a foundry like TSMC for masking⁵⁶.

Given the exorbitant financial cost of the physical mask set, engineers employ a vital risk-mitigation strategy involving "spare gates." During the physical placement phase, uncommitted logic gates (NANDs, NORs, multiplexers) are sprinkled uniformly across the floorplan⁶⁵. If a minor functional bug is discovered post-tapeout during initial silicon bring-up, it

can often be fixed by rewiring the interconnects to utilize these spare gates. Because the underlying base transistors are not altered, only the upper metal routing layers need to be modified. This process, known as a Metal ECO (Engineering Change Order), costs a mere fraction of a full mask respin and drastically reduces the time required to yield functional production silicon⁶⁵.

6.4 The Middle Ground: Structured ASICs

For projects where the projected volume is too high to justify recurring FPGA unit costs, but the massive NRE of a 5nm standard-cell ASIC is deemed too financially risky, the industry offers a transitional vehicle: Structured ASICs (e.g., Intel eASIC or TSMC/Faraday solutions)⁴².

A structured ASIC provides a predefined base array of logic cells, memory blocks, and I/O transceivers. The customer only customizes the top few metal routing layers to define the interconnects⁶⁹. Because the foundry pre-fabricates the complex lower silicon layers (the front-end-of-line) in bulk, the NRE drops drastically (often to \$100k–\$500k), and the development time shrinks from 18 months to a mere 3–6 months⁴². While structured ASICs consume more area and power than a full-custom standard cell design, they provide a highly effective stepping stone for transitioning hardened FPGA spatial dataflow architectures into silicon with substantially reduced financial risk⁴².

7. Conclusion

The architectural migration of AI workloads from general-purpose GPUs to custom spatial architectures is fundamentally driven by the immutable physics of memory bandwidth and power efficiency at a batch size of one. By leveraging the MLIR compiler framework, utilizing CIRCT, and systematically dismantling the SIMT assumptions of GPU languages like Triton, engineers can successfully recompile high-level algorithms into streaming, dataflow-driven RTL.

Through the use of persistent on-chip states and highly optimized ping-pong buffering, FPGAs achieve initiation intervals that completely eradicate the memory-bound limits of modern LLM decoding. However, committing these powerful spatial architectures to a full ASIC tapeout requires an uncompromising methodology. It demands exhaustive formal verification with SVA, rigorous UVM-based dynamic testing against PyTorch reference models, and high-speed Palladium emulation to validate software integration. By methodically addressing the physical realities of IR drop, DFT timing penalties, and strategically utilizing safeguards like metal ECOs, design teams can confidently cross the economic chasm from FPGA prototyping to high-volume, highly efficient ASIC deployment.

Works cited

1. FPGA vs GPU: Which Is Better for AI and High-Performance, <https://www.ampheo.com/blog/fpga-vs-gpu-which-is-better-for-ai-and-high-performance-computing>
2. A Persistent-State Dataflow Accelerator for Memory-Bound Linear, <https://arxiv.org/pdf/2603.05931>

3. A Hybrid Systolic-Dataflow Architecture for Inductive Matrix Algorithms,
<http://www.seas.ucla.edu/~jianw/hpca2020.pdf>
4. Transmuter: Bridging the Efficiency Gap using Memory and Dataflow,
<https://blaauw.engin.umich.edu/wp-content/uploads/sites/342/2021/02/Transmuter-Bridging-the-Efficiency-Gap-using-Memory-and-Dataflow-Reconfiguration.pdf>
5. (PDF) StreamTensor: Make Tensors Stream in Dataflow,
https://www.researchgate.net/publication/395582862_StreamTensor_Make_Tensors_Stream_in_Dataflow_Accelerators_for_LLMs
6. Make Tensors Stream in Dataflow Accelerators for LLMs - Hanchen Ye,
https://hanchenye.com/assets/pdfs/MICRO25_StreamTensor.pdf
7. Scalable Equivalence Checking for Behavioral Synthesis,
https://pdxscholar.library.pdx.edu/cgi/viewcontent.cgi?article=3467&context=open_access_etds
8. Streaming Data Paradigm - 2026.1 English - UG1399,
<https://docs.amd.com/r/en-US/ug1399-vitis-hls/Streaming-Data-Paradigm>
9. politecnico di torino, <https://webthesis.biblio.polito.it/37699/1/tesi.pdf>
10. Hardware Acceleration for Neural Networks: A Comprehensive Survey,
<https://arxiv.org/html/2512.23914v1>
11. A Persistent-State Dataflow Accelerator for Memory-Bound Linear,
<https://arxiv.org/html/2603.05931v1>
12. A Persistent-State Dataflow Accelerator for Memory-Bound Linear,
<https://www.alphaxiv.org/abs/2603.05931>
13. FPGA-Powered AI Inference Server Hardware for Long-Context,
<https://marketintel.com/report/fpga-powered-ai-inference-server-hardware-for-long-context-transformers-market>
14. fpga | Open Research Institute, <https://www.openresearch.institute/tag/fpga/>
15. FPGA AI Accelerators in VLSI Design | PDF - Scribd,
<https://www.scribd.com/document/942652839/FPA-AI-Accelerators-1761788289>
16. SOLUTION CHALLENGE Up to 3X TOPS/Watt - AMD,
<https://www.amd.com/content/dam/amd/en/documents/products/adaptive-socs-and-fpgas/versal/versal-gen2-airborne-solution-brief.pdf>
17. Real-Time FPGA-Based Transformers & VLMs for Vision Tasks - arXiv,
<https://arxiv.org/pdf/2509.04162>
18. Triton for MTIA: Bridging the Programming Model Gaps for Custom,
<https://arxiv.org/pdf/2608.00325>
19. Design Methodologies for Deep Learning Accelerators on ... - arXiv,
<https://arxiv.org/html/2311.17815v3>
20. Democratizing AI Accelerators and GPU Kernel Programming using,
<https://next.redhat.com/2024/11/07/democratizing-ai-accelerators-and-gpu-kernel-programming-using-triton/>
21. Generating Fast GPU Kernels without Programming in CUDA/Triton,
<https://zhihaojia.medium.com/generating-fast-gpu-kernels-without-programming-in-cuda-triton-3fdd4900d9bc>
22. ScaleHLS: A New Scalable High-Level Synthesis Framework on,
https://www.researchgate.net/publication/360662088_ScaleHLS_A_New_Scalable

- [High-Level Synthesis Framework on Multi-Level Intermediate Representation](#)
23. MING: An Automated CNN-to-Edge MLIR HLS framework - arXiv,
<https://arxiv.org/html/2602.11966v1>
 24. (PDF) LLVM-Based Compiler Architecture for Radiology AI Systems,
https://www.researchgate.net/publication/404788183_LLVM-Based_Compiler_Architecture_for_Radiology_AI_Systems_Technical_Research_Report_High-Performance_Medical_Imaging_Pipeline_Initiative_LLVM-Based_Compiler_Architecture_for_Radiology_AI
 25. ScaleHLS: A New Scalable High-Level Synthesis Framework on,
https://hanchenye.com/assets/pdfs/HPCA22_ScaleHLS.pdf
 26. ScaleHLS: a Scalable High-Level Synthesis Framework with Multi,
https://hanchenye.com/assets/pdfs/DAC22_ScaleHLS.pdf
 27. An MLIR-Based High-Level Synthesis Compiler for Hardware,
<https://hicaps.csa.iisc.ac.in/theses/thesis.pdf>
 28. HEC: Equivalence Verification Checking for Code Transformation,
<https://arxiv.org/html/2506.02290v1>
 29. Chisel 与 CIRCT 的无缝集成 - 兆松科技,
<https://www.terapines.com/articles/2024/11/29/Chisel-%E4%B8%8E-CIRCT-%E7%9A%84%E6%97%A0%E7%BC%9D%E9%9B%86%E6%88%90/>
 30. Compiler Technologies in Deep Learning Co-Design: A Survey,
https://www.researchgate.net/publication/371191540_Compiler_Technologies_in_Deep_Learning_Co-Design_A_Survey
 31. Tutorial @ DATE 2025 - Yun (Eric) Liang,
<https://ericlyun.me/tutorial-date2025/rsrc/ahs.final.pdf>
 32. Hands-on Schedule - ASP-DAC,
<https://www.aspdac.com/aspdac2026/archive/pdf/T5.pdf>
 33. An FPGA-Based LOCO-ANS Implementation for Lossless and Near,
<https://www.mdpi.com/2079-9292/10/23/2934>
 34. Adaptive Resource Allocation for Improving Energy Efficiency in,
https://research.chalmers.se/publication/537427/file/537427_Fulltext.pdf
 35. Compiler Design for Accelerating Applications on Coarse-Grained,
<https://mpslab-asu.github.io/publications/papers/Balasubramanian2021THESIS.pdf>
 36. A Comprehensive Methodology to Optimize FPGA Designs via the,
https://escholarship.org/content/qt6m06x3xv/qt6m06x3xv_noSplash_acb0be4341a27c0272774191c7ef6e95.pdf
 37. FPGA Engineer Interview Questions - Startup Jobs,
<https://startup.jobs/interview-questions/fpga-engineer>
 38. HDL vs HLS : r/FPGA - Reddit,
https://www.reddit.com/r/FPGA/comments/nc95js/hdl_vs_hls/
 39. Emerging Applications of FPGAs and Reconfigurable Computing,
https://mdpi-res.com/bookfiles/book/12935/Emerging_Applications_of_FPGAs_and_Reconfigurable_Computing_System.pdf?v=1788854836
 40. ATLAS: Automated HLS for DL-Optimized FPGAs - ResearchGate,
https://www.researchgate.net/publication/408622693_ATLAS_Automated_HLS_for

[r_DL-Optimized_FPGAs](#)

41. How to Improve Your FPGA Programming Skills: 8 Practices ... - Scaler,
<https://www.scaler.com/blog/how-to-improve-your-fpga-programming-skills/>
42. AI ASIC vs FPGA: Which Is Better? - Go Electronic,
<https://goelectronic.org/ai-asic-vs-fpga-which-is-better/>
43. (PDF) A Guide to Cost-Effective ASIC Design with Open-Source,
https://www.researchgate.net/publication/405401713_A_Guide_to_Cost-Effective_ASIC_Design_with_Open-Source_Tools_Business_Case_for_New_Start_Ups_in_2026
44. All About Formal Verification Jobs in the VLSI Industry - Inskill,
<https://inskill.in/training/vlsi/formal-verification-jobs-in-vlsi/>
45. Formal Verification: An Essential Toolkit for Modern VLSI Design,
https://www.researchgate.net/publication/280720663_Forma_Verification_An_Essential_Toolkit_for_Modern_VLSI_Design
46. Formal Verification AI for Semiconductor Correctness | Veriprajna,
<https://veriprajna.com/technical-whitepapers/semiconductor-ai-hardware-correctness>
47. Formal Property Verification Course - VLSIGuru,
<https://vlsiguru.com/short-term-courses/formal-property-verification>
48. Top 10 Silicon Chip Design Engineer Interview Questions and,
<https://www.interviews.chat/questions/silicon-chip-design-engineer>
49. Using Python in Verification - Alpinum Consulting,
<https://alpinumconsulting.com/wp-content/uploads/PythonAndVerification-v2.pdf>
50. design-verification · GitHub Topics,
<https://github.com/topics/design-verification?l=systemverilog&o=desc&s=updated>
51. GEM: GPU-Accelerated Emulator-Inspired RTL Simulation - Yibo Lin,
https://yibolin.com/publications/papers/SIM_DAC2025_Guo.pdf
52. Emulation Detailed Guide | PDF | Emulator | Palladium - Scribd,
<https://www.scribd.com/document/1052854058/Emulation-Detailed-Guide>
53. CNDS Palladium is #1 Best of 2019 - DeepChip,
<http://www.deepchip.com/items/dac19-01b.html>
54. Emulation-based System-on-Chip Security Verification - arXiv,
<https://arxiv.org/html/2604.15073v1>
55. What Is IR Drop and How to Fix It? – VLSI Power Integrity 2026,
<https://vlsiguru.com/blog/what-is-ir-drop-and-how-to-fix-it-vlsi>
56. ASIC Design Flow From Spec to Tape Out in Semiconductor,
<https://chippedge.com/resources/asic-design-flow-specification-to-tape-out-semiconductor/>
57. Physical Design Engineers - Game 7 Staffing,
<https://www.game7staffing.com/disciplines/physical-design>
58. The Ultimate Signoff (TapeOut) Checklist - AnySilicon,
<https://anysilicon.com/the-ultimate-signoff-tapeout-checklist/>
59. Follow these guidelines to design testable ASICs, boards, and,
<http://cleanmetrics.com/pages/EDN93-TestableDesign.pdf>
60. Expert Synthesis & STA Services - MiraFra Technologies,

- <https://mirafra.com/synthesis-and-sta/>
61. Design for Testability: Scan Chain Architectures and Variants,
<https://medium.com/@ranaumarnadeem/design-for-testability-scan-chain-architectures-and-variants-623ea712fa52>
 62. Reducing DFT Footprints: A Case in Consumer SoC - eInfochips,
<https://www.einfochips.com/blog/reducing-dft-footprints-a-case-in-consumer-soc/>
 63. ADVANCED ASIC CHIP SYNTHESIS - ResearchGate,
https://www.researchgate.net/profile/Mohan_Pradhan/post/What_is_the_advantages_of_using_Synopsys_Physical_Compiler_over_Design_compiler_in_Logic_Synthesis/attachment/59d6279579197b8077985e0a/AS:326477930352640@1454849812435/download/ASIC_chip_synthesis.pdf
 64. HBD872 - onsemi, <https://www.onsemi.com/pub/collateral/hbd872-d.pdf>
 65. Cadence Innovus User Guide v23.14 - IC Physical Design & EDA,
<https://studylib.net/doc/28007780/innovusug>
 66. ASIC Design Implementation Framework | PDF - Scribd,
<https://www.scribd.com/document/975305381/ASIC-Design-Implementation-Process-A-Complete-Framework-Khosrow-Golshan-WeLib-org>
 67. VLSI Physical Design Complete Guide | PDF - Scribd,
<https://www.scribd.com/document/1065477713/VLSI-Physical-Design-Complete-Guide>
 68. 2023 Faraday Sustainability Report,
https://www.faraday-tech.com/resources/PDFFile/CSR/Faraday_ESG_Report_2023_en.pdf
 69. US7598766B2 - Customized silicon chips produced using,
<https://patents.google.com/patent/US7598766>
 70. Application-Specific Integrated Circuit: What Is an ASIC? - Articsledge,
<https://www.articsledge.com/post/application-specific-integrated-circuit-asic>
 71. ON Semiconductor Delivers Structured ASIC Technology for Military,
<https://www.design-reuse.com/news/202514723-on-semiconductor-delivers-structured-asic-technology-for-military-and-aerospace-socs-and-fpga-to-asic-conversions/>